

SIT720 Week 8.2 — Melbourne Housing Price Prediction

This write-up explains what I did across the five required steps: data acquisition, preprocessing and EDA, model development, feature importance, and deployment.

Step 1. Melbourne Housing Data Acquisition

I started by choosing three suburbs I'd consider living in: Highton (VIC 3216), Ballarat (Greater Region, VIC), and Werribee (VIC 3030). I then collected housing data from realestate.com.au.

- I built a small scraper (`housing_data_scraper.py`) using Selenium + BeautifulSoup. It navigates suburb result pages (up to four per suburb) and extracts: suburb, address, property_type, bedrooms, bathrooms, parking, land_size, sold_price, sold_date, listing_url.
- I added optional cookie loading (`www.realestate.com.au.cookies.json`) to stabilize the session.
- After scraping, I saved the dataset to `melbourne_housing_data.csv`.

For this analysis, I used a clean sample of 300 rows across Highton, Ballarat, and Werribee (≥50 per suburb satisfied overall; the final modeling dataset contains 300 records in total).

Step 2. Data Preprocessing and Exploratory Data Analysis (EDA)

I prepared the data in `task.ipynb` as follows:

- Parsed and sorted dates: I converted `sold_date` to datetime and sorted records by date (newest first) so time-aware plots make sense.
- One-hot encoding: I encoded suburb and property_type to create columns like `suburb_Highton` and `property_type_House`.
- Feature engineering (leak-free): I engineered useful features that don't use the target, including:
 - `land_per_bedroom = land_size / max(bedrooms, 1)`
- Scaling and missing values: I prepared standardized copies of numeric features for EDA. Models were trained on the original scale. I filled parking with 0 and imputed `land_size` by median during model prep. Any infinities from engineered ratios were converted to NaN and median-filled.
- EDA visuals:
 - Distribution of `sold_price` (histogram + KDE) and a price box plot.
 - Correlation heatmap for numeric features.
 - Outlier inspection with box plots.
 - Price trends over time per suburb (line plot of average sale price).

Step 3. Model Development, Metrics, and Cross-Validation

I trained three regressors using scikit-learn, with an 80/20 train/test split (`random_state=42`):

- Linear Regression
- Random Forest Regressor (`n_estimators=100`, `random_state=42`)
- Gradient Boosting Regressor (`n_estimators=100`, `random_state=42`)

I evaluated them with MAE, RMSE, and R^2 . Representative test-set results from the notebook are:

- Linear Regression: MAE $\approx$ 93,856 | RMSE $\approx$ 129,883 | $R^2 \approx$ 0.701
- Random Forest: MAE $\approx$ 111,059 | RMSE $\approx$ 178,204 | $R^2 \approx$ 0.437
- Gradient Boosting: MAE $\approx$ 108,373 | RMSE $\approx$ 164,434 | $R^2 \approx$ 0.521

I also ran 5-fold cross-validation, which supported the same ranking:

- Linear Regression: MAE $\approx$ 111,723 | RMSE $\approx$ 167,769 | $R^2 \approx$ 0.549
- Random Forest: MAE $\approx$ 110,911 | RMSE $\approx$ 177,869 | $R^2 \approx$ 0.501
- Gradient Boosting: MAE $\approx$ 112,218 | RMSE $\approx$ 173,329 | $R^2 \approx$ 0.529

Overall, Linear Regression generalized best on the test set for this dataset snapshot, while tree models fit the training data more strongly but generalized less without tuning.

Step 4. Feature Importance

To understand what drives prices, I inspected feature importance in multiple ways:

- Linear Regression coefficients (by absolute value) highlighted bathrooms, bedrooms, and location/type indicators (for example, `suburb_Highton`, `property_type_*`).
- Random Forest and Gradient Boosting feature importances consistently ranked `land_size` at or near the top, followed by bathrooms, bedrooms, and suburb effects.

These results align with intuition: land size and basic room counts are strong predictors, and both suburb and property type matter.

Step 5. Model Deployment (Web Demo)

I created a simple Gradio interface inside the notebook to make predictions from user inputs.

- Served model: I exposed the Linear Regression model (the best generalizer in my runs).
- Inputs: `suburb`, `property_type`, `bedrooms`, `bathrooms`, `parking`, and `land_size` (falling back to the median if unknown).
- Output: a currency-formatted price prediction.

- To run it: in `task.ipynb`, install requirements with `%pip install -q gradio` if needed, then uncomment `iface.launch(share=True)` in the deployment cell and execute it. The app starts locally and can optionally create a shareable URL.